

SEQUENCES, CONTAINERS, OBJECTS, & LINKED LISTS Meta

COMPUTER SCIENCE MENTORS 61A

September 28, 2026 – October 2, 2026

Example Timeline

- **This is a menu of problems, not a requirement to finish the worksheet.** Individual suggested times do not add up to one hour. Choose or skip problems based on your students, and give each mini-lecture immediately before its section.
- Check-in and icebreaker [5 min]. Ask what students would like more practice with: tracing expressions, choosing a representation, or writing code.
- Sequences: selected parts of Q1–Q2, then Q3 or Q4 [15 min]. Choose Q3 for nested loops or Q4 for filtering and comprehensions. Q5 is extra practice.
- Containers: Q6 or Q7 [10 min]. Choose Q6 for strings and counting or Q7 to connect dictionaries to higher-order functions.
- Objects and abstraction: Q8 [5 min]. Use the supplied linked-list class as the concrete object example. Q9 is optional practice if students have already seen methods and inheritance.
- Linked lists: the first block of Q10, then Q11 or Q12 [20 min]. Choose Q11 for construction or Q12 for processing an existing linked list. Save cycles, representations, and the iterative version for students ready for more.
- Wrap-up and questions [5 min]. Have students describe the interface of one representation in their own words.

Teaching Goals

- Keep emphasizing abstraction: what information does a value represent, and which operations does its interface support? Have students name intermediate values and their types before evaluating nested expressions.
- Let students predict, discuss with a partner, and explain their reasoning before running code. For implementation problems, establish an example and a plan before filling in blanks.
- Q1 includes mutation and shallow copying as well as indexing. Prioritize the indexing and nested-list expressions; treat the later mutation cases as an extension if needed.
- **Container type hints (Q2, Q3, Q6):** Before solving, have students read the annotations aloud. A tuple annotation lists the types at each position; `list[int]` describes integer elements; `dict[str, int]` describes string keys and integer values. The arrow describes a function's return type. Annotations document expected types; Python does not automatically enforce them.
- The sequence overview and Q2 introduce tuples and their operations. Q10–Q12 cover linked-list structure, construction, traversal, and processing.
- Q13 is optional tree-recursion practice: split the possibilities into taking one donut of the current flavor or using none of that flavor. Ask students why these two counts are added.

Sequence Overview

Sequences are ordered data structures that have length and support element selection. Here are some common types of sequences you'll be dealing with in this class:

- Lists: `[1, [2], 'a', lambda x: 5]`
- Tuples: `(1, (2,), 'a', lambda x: 5)`
- Strings: `'Hello World!'`

While each type of sequence is different, they all share a common interface for manipulating and accessing their data:

- **Item selection:** Use square brackets to select an element at an index:

```
(3, 1, 2)[0] # returns 3
"Hello"[-1]  # returns "o"
```

- **Length:** The built-in `len` function returns the length of a sequence:

```
len((1, 2)) # returns 2
```

- **Concatenation:** Sequences can be concatenated with the `+` operator, which returns a *new* sequence:

```
[1, 2] + [3, 4] # returns [1, 2, 3, 4]
```

- **Membership:** The `in` operator tests for sequence membership:

```
1 in (1, 2, 3)      # returns True
5 not in (1, 2, 3)   # returns True
"apple" in "snapple" # returns True
```

Membership in Strings vs. Lists and Tuples: As a short aside, while the `in` operator works the same for lists and tuples, checking if an element is contained within the list/tuple container, the `in` operator instead for strings checks for direct substrings rather than the existence of distinct elements within the string.

- **Looping:** Sequences can be looped through with `for` loops:

```
>>> for x in [1, 2, 3]:
...     print(x)
1
2
3
```

- **Aggregation:** Common built-in functions—including `sum`, `min`, and `max`—can take sequences and aggregate them into a single value:

```
max((3, 4, 5)) # returns 5
```

- **Slicing:** Slicing is a way to create a copy of all or part of a sequence. The general syntax for slicing a sequence `seq` is as follows:

```
seq[<start index>:<end index>:<step size>]
```

This evaluates to a new sequence that includes every element starting at `<start index>` and up to and *excluding* `<end index>` in `seq`, taking steps of size `<step size>`.

If we do not supply `<start index>` or `<end index>`, it will start at the beginning of the sequence and include every element up to and including the end of the sequence.

```

>>> lst = [1, 2, 3, 4, 5]
>>> lst[2:]
[3, 4, 5]
>>> lst[:3]
[1, 2, 3]
>>> lst[::-1]
[5, 4, 3, 2, 1]
>>> lst[1:2]
[2, 4]

```

List comprehensions create a new list by processing an iterable, such as a list, tuple, string, or range. The syntax for a list comprehension is

```
[<expression> for <element> in <sequence> if <condition>]
```

We could equivalently write the following:

```

lst = []
for <element> in <sequence>:
    if <condition>:
        lst = lst + [<expression>]

```

The **if** <condition> filter statement is optional. The following list comprehension doubles each odd element of [1, 2, 3, 4]:

```

>>> [i * 2 for i in [1, 2, 3, 4] if i % 2 != 0]
[2, 6]

```

Equivalent in **for** loop syntax:

```

lst = []
for i in [1, 2, 3, 4]:
    if i % 2 != 0:
        lst = lst + [i * 2]

```

A lot of information in this guide is not the full and complete picture of how Python works, but students don't need to know that. Often a little misdirection is necessary to improve initial knowledge acquisition before a more complete picture is painted later on. I would just be sure to mention the nature of sequences: a loose umbrella (not a strict definition like a class) that encompasses many different data types.

If students are confused consider bringing up specific examples of sequences. Reading rules on a page is often not actually that instructive.

Sequence Practice

1. What would Python display? Draw box-and-pointer diagrams for the following:

```
>>> a = [1, 2, 3]
>>> a
```

```
[1, 2, 3]
```

```
>>> a[2]
```

```
3
```

```
>>> a[-1]
```

```
3
```

```
>>> b = a
>>> a = a + [4, [5, 6]]
>>> a
```

```
[1, 2, 3, 4, [5, 6]]
```

```
>>> b
```

```
[1, 2, 3]
```

```
>>> c = a
>>> a = [4, 5]
>>> a
```

```
[4, 5]
```

```
>>> c
```

```
[1, 2, 3, 4, [5, 6]]
```

```
>>> d = c[3:5]
>>> c[3] = 9
>>> d
```

```
[4, [5, 6]]
```

```
>>> c[4][0] = 7
>>> d
```

```
[4, [7, 6]]
```

```
>>> c[4] = 10
>>> d
```

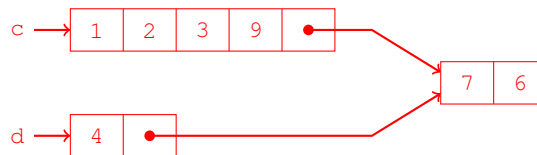
```
[4, [7, 6]]
```

```
>>> c
```

```
[1, 2, 3, 9, 10]
```

Shared Nested List

Immediately after `c[4][0] = 7`, the lists referenced by `c` and `d` share the same inner list. Only these two variables are shown.



The next assignment, `c[4] = 10`, replaces the reference in `c`; `d[1]` still points to `[7, 6]`.

[Click this text and share this link with students; this link renders Python Tutor diagram of this question.](#)

How to Draw Lists

Draw one box per list element and label the indices starting at zero. Draw arrows from variable names to their list objects, and from list elements to any nested lists. When two arrows reach the same object, changing that object is visible through both references. In this question, contrast creating a new outer list with changing a shared nested list. Use the solution diagram and Python Tutor link after students have made their predictions.

Teaching Tips

1. Draw box-and-pointer diagrams on the board: use one box per list element and arrows for references to nested lists.
2. Encourage students to draw box and pointer diagrams if they seem stuck
3. It can be helpful to visually go through indexing and list slicing in the box and pointer diagram

4. Python is confusing when adding on to a list with `lst += [element]` and `lst = lst + element`. The former mutates and the latter creates a new list
5. Make sure you clearly state when you're making a new list object (i.e. at `a = a + [4, [5, 6]]` and list slicing like `d = c[3:5]`)
6. Be sure to touch on negative indices and reiterate the difference between shallow and deep copies – as it's a nuance in list comprehension, we decided not to include it in the overview, but when going over this problem, please make sure to touch on this!
7. If students need additional help with shallow and deep copying, feel free to come up with your own creative iterations on the shallow and deep copying problems.

Suggested Time: 8–12 min; Difficulty: Easy–Medium Prioritize positive, negative, and nested indexing. Treat later mutation cases as an extension. Distinguish rebinding a name from changing a list; slices make a new outer list with references to the same elements.

2. **Tuples.** What would Python display? Run these statements in order. If a statement causes an error, identify the error and continue with the next statement. For `(3,)` and `(3)`, also name the type.

Creating tuples

```
>>> t: tuple[int, str, float, bool, None] = (
...     3, "csm", 2.5, True, None
... )
>>> t
```

```
(3, 'csm', 2.5, True, None)
```

```
>>> 4, 5, 6, 7
```

```
(4, 5, 6, 7)
```

```
>>> (3,)
```

```
(3,)
```

```
Type: tuple.
```

```
>>> (3)
```

```
3
```

```
Type: int.
```

```
>>> ()
```

```
>>> len(())
```

```
()
```

```
0
```

Slicing and converting

```
>>> t[1:4]

('csm', 2.5, True)

>>> t[::-1]

(None, True, 2.5, 'csm', 3)

>>> list(t)

[3, 'csm', 2.5, True, None]

>>> list("csm")
>>> list(())

['c', 's', 'm']
[]
```

Combining tuples

```
>>> (4, 5) + (6, 7)

(4, 5, 6, 7)

>>> (3,) * 3
>>> (3) * 3

(3, 3, 3)
9

>>> t[0] = 4
```

`TypeError`

Dictionary keys

```
>>> locations = {(4, 5): "CSM"}
>>> locations[(4, 5)]

'CSM'
```

```
>>> locations[[4, 5]] = "office hours"
```

`TypeError`

```
>>> locations[(4, [5])] = "office hours"
```

`TypeError`

Suggested Time: 8–10 min; Difficulty: Easy–Medium

- Ask students to read the annotation on `t`: it describes a five-element tuple, with the type of each position listed in order. The annotation does not convert values or enforce their types at runtime.
 - A comma creates a tuple: parentheses usually group expressions. The empty tuple is written `()`; one item needs a trailing comma.
 - Tuples can hold different data types. Slicing returns a tuple; **list** builds a list from the elements of an iterable.
 - Tuple addition concatenates and multiplication repeats elements; neither operation changes the original tuple. Tuple item assignment fails.
 - Dictionary keys must be hashable. A tuple can be a key only when all its elements are hashable. Lists are unhashable, including a list nested inside a tuple. Use the last example as a follow-up challenge.
3. Write a function `duplicate_list`, which takes in a list of positive integers and returns a new list with each element `x` in the original list duplicated `x` times.

```
def duplicate_list(lst: list[int]) -> list[int]:  
    """  
    >>> duplicate_list([1, 2, 3])  
    [1, 2, 2, 3, 3, 3]  
    >>> duplicate_list([5])  
    [5, 5, 5, 5, 5]  
    """
```

```
    for _____:
```

```
        for _____:
```

```
            _____
```

```
    _____
```

```
    new_list = []  
    for x in lst:  
        for i in range(x):  
            new_list = new_list + [x]  
    return new_list
```

Teaching Tips

1. Read the signature before solving: `list[int]` describes a list of integers, and `-> list[int]` describes the expected return type. Type hints do not enforce types at runtime.
2. If students have trouble arriving at the solution, walk through the intuition of nested for loops and discuss what each loop represents. For example, the first loop represents iterating over each element of the list and the second one represents repeating that element.
3. This is a good problem to emphasize how we can format our logic and approach to problems based on the skeleton code.

Suggested Time: 7 min; Difficulty: Medium Explain what each loop represents: the outer loop selects an element, and the inner loop repeats it. Connect `range(x)` to exactly `x` repetitions.

4. Write a function that takes in a list `nums` and returns a new list with only the primes from `nums`. Assume that `is_prime(n)` is defined. You may use a **while** loop, a **for** loop, or a list comprehension.

```
def all_primes(nums):
```

```
    result = []
    for i in nums:
        if is_prime(i):
            result = result + [i]
    return result

List comprehension:
return [x for x in nums if is_prime(x)]
```

Teaching Tips

1. Students may be very familiar with list comprehensions at this point, so don't spend too much time going over this question if your students don't need it
2. If they are stuck doing it with list comprehensions, state that one can approach it with a for loop
3. Might help to go through how to convert from a for/while loop to a list comprehension
4. If students are struggling with list comprehensions, it might be a good idea to explain that the entire list comprehension returns a list, instead of just acting like a for loop.

Suggested Time: 5 min; Difficulty: Easy Treat `is_prime` as a provided abstraction. Focus on filtering a sequence, then compare the loop and comprehension solutions already supplied in the bank.

5. Fill in the methods below according to the doctests.

```
def gen_list(n):
    """
    Returns a nested list structure of n elements where the
    ith element is a list from 0 (inclusive) to i (exclusive).
    >>> gen_list(3)
    [[0], [0, 1], [0, 1, 2]]
    >>> gen_list(5)
    [[0], [0, 1], [0, 1, 2], [0, 1, 2, 3], [0, 1, 2, 3, 4]]
    """
    return _____
```

For an additional challenge, try out the following:

```
def gen_increasing(n):
    """
    Returns a nested list structure of n elements where the
    ith element of each list is one more than the previous
    element (even if the previous is in a prior sublist).
    >>> gen_increasing(3)
    [[0], [1, 2], [3, 4, 5]]
    >>> gen_increasing(5)
    [[0], [1, 2], [3, 4, 5], [6, 7, 8, 9], [10, 11, 12, 13,
    14]]
    """
    return _____
```

Hint: You can sum ranges. E.g. `sum(range(3))` gives us $0 + 1 + 2 = 3$.

```
def gen_list(n):
    return [[i for i in range(j+1)] for j in range(n)]

def gen_increasing(n):
    return [[i for i in range(sum(range(j+1)), sum(range(j+1)) + j+1)] for
            j in range(n)]
```

An alternate solution for `gen_increasing` is shown below:

```
def gen_increasing(n):
    return [[i + sum(range(j + 1)) for i in range(j + 1)] for j in
            range(n)]
```

Suggested Time: 7 min; add 5 min for the challenge; Difficulty: Medium Build one inner list before reasoning about the outer comprehension. The doctests number the elements starting at one. In the challenge, `sum(range(j + 1))` counts how many values earlier sublists contain. The wrapped final doctest output is one list, not separate outputs.

2 Containers and Dictionaries

6. Write a function `count_t`, which takes in a dictionary `d` and a string `word`. The function should count the instances of the letter "t" in `word` and add a key-value pair to the dictionary. The key will be `word` and the value will be the number of "t"s in `word`

```
def count_t(d: dict[str, int], word: str) -> None:
    """
    >>> words = {}
    >>> count_t(words, "tatter")
    >>> words["tatter"]
    3
    >>> count_t(words, "tree")
    >>> words
    {'tatter': 3, 'tree': 1}
    """
```

```
    for _____:
        if _____:
```

```
count = 0
for c in word:
    if c == 't':
        count += 1
d[word] = count
```

Teaching Tips

Suggested Time: 6 min **Difficulty:** Easy

- **Read the type hints.** `dict[str, int]` means string keys and integer values; `word: str` means a string input. Annotations describe expected types; Python does not automatically enforce them.
- **Review dictionary access.** A dictionary maps keys to values. Use `d[word]` to read a value and `d[word] = count` to add or update the entry.
- **Plan the loop.** Traverse the string directly with `for c in word`. Start the count at zero, increment it when `c == 't'`, and update the dictionary after the loop.
- **Distinguish mutation from returning.** The function changes `d` in place and implicitly returns `None`, matching `-> None`. Ask students why the doctests inspect `words` after the call.

7. Complete the function `snapshot`, which takes a single-argument function `f` and a list `snap_inputs` and returns a “snapshot” of `f` on `snap_inputs`. A “snapshot” is a dictionary where the keys are the provided `snap_inputs` and the values are the corresponding outputs of `f` on each input.

```
def snapshot(f, snap_inputs):  
    """  
    >>> snapshot(lambda x: x**2, [1, 2, 3])  
    {1: 1, 2: 4, 3: 9}  
    """  
  
    snap = _____  
  
    _____:  
        _____  
  
    return snap
```

```
def snapshot(f, snap_inputs):  
    snap = {}  
    for snap_input in snap_inputs:  
        snap[snap_input] = f(snap_input)  
    return snap
```

One way to think of a dictionary is as a function (in the mathematical sense) with a finite domain: you provide it with an input and it gives you some output. The idea behind this problem is to exercise that connection by having students convert between functions defined by rules that often have unlimited domains (e.g. $f(x) = x^2$) and finite functions that are defined by directly spelling out the outputs of a function. That’s why this problem is called “snapshot”—it’s a small snapshot of a function’s behavior over a limited domain.

Suggested Time: 7 min; Difficulty: Easy–Medium Connect this to functional abstraction from earlier weeks: the function is an input, and the dictionary records its outputs on selected inputs. Identify the key and value before filling in the assignment. The supplied solution uses a loop, not a dictionary comprehension.

3 Objects and Data Abstraction

8. (a) What is an **Abstract Data Type**?

A structure that stores data but separates how it is stored from how it is used.

There are two different layers to abstract data types:

1) The program layer, which uses the data, and 2) The concrete data representation that is independent of the programs that use the data. The only communication between the two layers is through selectors and constructors that implement the abstract data in terms of the concrete representation.

In addition to selectors and constructors, there are behavior conditions under which the selectors and constructors give an appropriate response. That is, if we construct a rational number data type x from integers n and d , then it should be the case that $\text{numer}(x) / \text{denom}(x)$ is equal to n / d .

In general, we can think of an abstract data type as defined by some collection of selectors and constructors, together with some behavior conditions. As long as the behavior conditions are met (such as the division property above), these functions constitute a valid representation of the data type.

(b) What is the relationship between a class and an ADT?

Classes are a way to implement an Abstract Data Type. But, ADTs can also be created using a collection of functions, as shown by the rational number example.

Suggested Time: 5 min; Difficulty: Conceptual Treat this as a conceptual discussion before moving on to code. Connect the interface/representation distinction to lists, dictionaries, and linked-list classes. A class is one way to implement an abstraction; it is not the definition of abstraction itself.

9. What would Python display?

```
class Foo(object):
    x = 'bam'

    def __init__(self, x):
        self.x = x

    def baz(self):
        return type(self).x + self.x

class Bar(Foo):
    x = 'boom'

    def __init__(self, x):
        Foo.__init__(self, 'er' + x)

foo = Foo('boo')
```

(a) `>>> Foo.x`

`'bam'`

(b) `>>> foo.x`

`'boo'`

```

(c) >>> foo.baz()
      'bamboo'
(d) >>> Foo.baz()
      Error
(e) >>> Foo.baz(foo)
      'bamboo'

(f) >>> bar = Bar('ang')
      >>> Bar.x

      'boom'
(g) >>> bar.x
      'erang'
(h) >>> bar.baz()
      'boomerang'

```

Suggested Time: 8 min; Difficulty: Medium; Optional This existing problem includes class attributes, instance attributes, bound methods, and inheritance. Use the `Foo` parts for initial object practice; use the `Bar` parts only after inheritance has been introduced. Draw separate locations for class and instance attributes, and make the implicit `self` argument explicit when tracing calls.

For each of the following problems, assume linked lists are defined as follows:

```
class Link:
    empty = ()
    def __init__(self, first, rest=empty):
        assert rest is Link.empty or isinstance(rest, Link)
        self.first = first
        self.rest = rest

    def __repr__(self):
        if self.rest is not Link.empty:
            rest_repr = ', ' + repr(self.rest)
        else:
            rest_repr = ''
        return 'Link(' + repr(self.first) + rest_repr + ')'

    def __str__(self):
        string = '<'
        while self.rest is not Link.empty:
            string += str(self.first) + ' '
            self = self.rest
        return string + str(self.first) + '>'
```

10. What will Python output? Draw box-and-pointer diagrams along the way.

```
>>> a = Link(1, Link(2, Link(3)))
```

```
+---+---+ +---+---+ +---+---+
| 1 | --|->| 2 | --|->| 3 | / |
+---+---+ +---+---+ +---+---+
```

```
>>> a.first
```

```
1
```

```
>>> a.first = 5
```

```
+---+---+ +---+---+ +---+---+
| 5 | --|->| 2 | --|->| 3 | / |
+---+---+ +---+---+ +---+---+
```

```
>>> a.first
```

```
5
```

```
>>> a.rest.first
```

```
2
```

```
>>> a.rest.rest.rest.rest.first
```

```
Error: tuple object has no attribute rest (Link.empty has no rest)
```

```
>>> a.rest.rest.rest = a
```

```

      +---+---+ +---+---+ +---+---+
+-->| 5 | --|->| 2 | --|->| 3 | --|---+
|   +---+---+ +---+---+ +---+---+ |
|                                     |
+-----+

```

```
>>> a.rest.rest.rest.rest.first
```

```
2
```

```
>>> repr(Link(1, Link(2, Link(3, Link.empty))))
```

```
"Link(1, Link(2, Link(3)))"
```

```
>>> Link(1, Link(2, Link(3, Link.empty)))
```

```
Link(1, Link(2, Link(3)))
```

```
>>> str(Link(1, Link(2, Link(3))))
```

```
'<1 2 3>'
```

```
>>> print(Link(Link(1), Link(2, Link(3))))
```

```
<<1> 2 3>
```

Teaching Tips

- For assignment statements, Python will not print anything but still have them draw out what the linked list will look like
- Note that we are doing mutation here, so we are actually altering the object that was created in the first assignment.
 - Some students may have minimal exposure to mutating objects so try to emphasize this and make it obvious through diagrams.
- For the error, walk-through how to keep track of which rest corresponds to which object in the box and pointer diagram. ****Make sure they understand why calling rest a fourth time will give us an error (look back at the class definition)****
 - Abstraction:
 - * our last .rest is set to Link.empty
 - * Link.empty is not a Link objects — they do not have a .rest attribute

- Actual implementation:
 - * our last .rest is set to Link.empty
 - * Link.empty is not a Link objects — they do not have a .rest attribute
- Reassigning the last .rest to point back at the front always trips students up.
 - Make it clear that a is a pointer that points to the linked list. So we are trying to assign the last rest of a to point at what a points to, which is the beginning of the list. ****To test their understanding ask what would be different if we instead had****:
 - * a.rest.rest.rest = a.rest
 - a way to explain the assignment for this problem is to emphasize the “evaluation” of the RHS and the LHS
 - what is the value of a (a pointer). Really emphasize the implications of pointers here.
 - where are we putting a into? (the box that represents a.rest.rest.rest)
 - same for a.rest.rest.rest = a.rest. what is the value of a.rest? (still a pointer!)
 - Mention that this creates a cycle in the list

Suggested Time: 7 min for the first block; add 5 min for the rest; Difficulty: Easy–Medium Prioritize `first`, `rest`, and the empty sentinel. Each attribute access operates on the value produced by the preceding expression. The second block introduces a cycle and string representations; treat those as extensions. Draw arrows instead of mentally tracking repeated `rest` accesses.

11. Write a function `linkify` that, given a list `lst`, returns a linked list with the same values. You may assume that there will be no nested lists.

Try writing this function both recursively and iteratively.

```
def linkify_rec(lst):
    """
    >>> lst = [0, 1, 2, 3]
    >>> linkify_rec(lst)
    Link(0, Link(1, Link(2, Link(3))))
    """
    if _____:
        _____

    else:
        _____

def linkify_iter(lst):
    _____

    for _____:
        _____

    _____

def linkify_rec(lst):
    if not lst:
        return Link.empty
    else:
        return Link(lst[0], linkify_rec(lst[1:]))

def linkify_iter(lst):
    retVal = Link.empty
    for elem in lst[::-1]:
        retVal = Link(elem, retVal)
    return retVal
```

Suggested Time: 9 min recursive; add 5 min iterative; Difficulty: Medium Connect to the recursive leap of faith: identify the base case, the smaller input, and how to build the answer from the recursive result. State the recursive contract before filling blanks: the recursive call returns a linked list for the remaining Python list. Check the empty-input case. For iteration, build from right to left because each new node is placed at the front.

12. Write a function `combine_two`, which takes in a linked list of integers `lnk` and a two-argument function `fn`. It returns a new linked list where every two elements of `lnk` have been combined using `fn`.

```
def combine_two(lnk, fn):
    """
    >>> lnk1 = Link(1, Link(2, Link(3, Link(4))))
    >>> combine_two(lnk1, add)
    Link(3, Link(7))
    >>> lnk2 = Link(2, Link(4, Link(6)))
    >>> combine_two(lnk2, mul)
    Link(8, Link(6))
    """
    if _____:
        return _____
    elif _____:
        return _____
    combined = _____
    return _____

def combine_two(lnk, fn):
    if lnk is Link.empty:
        return Link.empty
    elif lnk.rest is Link.empty:
        return Link(lnk.first)
    combined = fn(lnk.first, lnk.rest.first)
    return Link(combined, combine_two(lnk.rest.rest, fn))
```

Suggested Time: 10 min; Difficulty: Medium The names `add` and `mul` in the doctests are the functions from Python's `operator` module. Separate empty, one-element, and longer inputs. Each recursive step consumes two elements and constructs one result node; an unpaired last element is retained. Connect the supplied function argument to HOFs and earlier practice with `combine`: both accept an operation as an argument, but this problem combines adjacent linked-list elements into a new linked list.

5 Optional Extension: Tree-Recursive Counting

13. Gabe's Donut shop has an unlimited supply of f different flavors of donuts. Adit wants to buy a box containing d donuts. Complete the skeleton for the function `donut`, which determines the number of possible ways there are for Adit to select his d donuts from the f flavors. You may assume that d and f are non-negative integers.

Hint: Does order matter?

```
def donut(d, f):  
    """  
    >>> donut(12, 1)  
    1  
    >>> donut(12, 2)  
    13  
    >>> donut(12, 12)  
    1352078  
    >>> donut(0, 0)  
    1  
    """  
    if _____:  
        return _____  
  
    if _____:  
        return _____  
  
    return _____  
  
def donut(d, f):  
    if d == 0:  
        return 1  
    if f == 0:  
        return 0  
    return donut(d - 1, f) + donut(d, f - 1)
```

Suppose the flavors are numbered 1 through f . Because order does not matter, we're going to say that Adit fills up his donut box starting with flavor number f and working down to flavor number 1. Under this scheme, Adit has a choice to make: does he want a donut of flavor number f , or does he not want one? If he does initially take a donut of flavor number f , then the number of ways to select the remaining $d - 1$ slots of the box is `donut(d - 1, f)`. If he does not want a donut of flavor number f , then he has d slots left in the box and $f - 1$ flavors to fill them with, making the number of ways to make the selection `donut(d, f - 1)`. Since he must make the choice one way or the other, the total number of ways to fill up the box is `donut(d - 1, f) + donut(d, f - 1)`.

If your students are confused on why the doctest `donut(0, 0)` returns 1, you can think about it this way:

if you want 0 donuts and they have no flavors, then there is one action you can take, do nothing and don't purchase donuts

A big concept within tree recursion counting problems is this idea that we can use some sort of “choice” to break our problem down into smaller subproblems. I would emphasize this heavily to my students. Here, the choice is whether or not Adit takes a certain flavor.

In this case, we have deliberately neglected to provide several base cases in the doc tests. While a good exam problem would include these doctests, they are often not provided, and students should be able to deduce the bases cases by either 1) mathematical maturity or 2) deduction from test inputs. In the latter case, they could for example note that `donut(1, 1) == donut(0, 1) + donut(1, 0)` and use this to help figure out what `donut` should return at these base cases.

Suggested Time: 10 min; Difficulty: Medium Taking a donut decreases the number of remaining slots; excluding a flavor decreases the available flavors. These cases do not overlap, so add their counts. Have students describe what each recursive call counts before filling in the code. A box with zero donuts is one completed choice, which explains the successful base case. Connect to earlier practice with `mario_number`: add the counts of separate choices. Ask how this differs from `make_change`, which chooses the minimum rather than counting all possibilities.